JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class MappedByteBuffer

java.lang.Object
    java.nio.Buffer
        java.nio.ByteBuffer
            java.nio.MappedByteBuffer

All Implemented Interfaces:

Comparable<ByteBuffer>

```
public abstract sealed class MappedByteBuffer
extends ByteBuffer
```

A direct byte buffer whose content is a memory-mapped region of a file.

Mapped byte buffers are created via the `FileChannel.map` method. This class extends the `ByteBuffer` class with operations that are specific to memory-mapped file regions.

A mapped byte buffer and the file mapping that it represents remain valid until the buffer itself is garbage-collected.

The content of a mapped byte buffer can change at any time, for example if the content of the corresponding region of the mapped file is changed by this program or another. Whether or not such changes occur, and when they occur, is operating-system dependent and therefore unspecified.

All or part of a mapped byte buffer may become inaccessible at any time, for example if the mapped file is truncated. An attempt to access an inaccessible region of a mapped byte buffer will not change the buffer's content and will cause an unspecified exception to be thrown either at the time of the access or at some later time. It is therefore strongly recommended that appropriate precautions be taken to avoid the manipulation of a mapped file by this program, or by a concurrently running program, except to read or write the file's content.

Mapped byte buffers otherwise behave no differently than ordinary direct byte buffers.

Since:

1.4

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
	`clear()`	Clears this buffer.

final MappedByteBuffer		
abstract MappedByteBuffer	**compact**()	Compacts this buffer *(optional operation)*.
abstract MappedByteBuffer	**duplicate**()	Creates a new byte buffer that shares this buffer's content.
final MappedByteBuffer	**flip**()	Flips this buffer.
final MappedByteBuffer	**force**()	Forces any changes made to this buffer's content to be written to the storage device containing the mapped file.
final MappedByteBuffer	**force**(int index, int length)	Forces any changes made to a region of this buffer's content to be written to the storage device containing the mapped file.
final boolean	**isLoaded**()	Tells whether or not this buffer's content is resident in physical memory.
final MappedByteBuffer	**limit**(int newLimit)	Sets this buffer's limit.
final MappedByteBuffer	**load**()	Loads this buffer's content into physical memory.
final MappedByteBuffer	**mark**()	Sets this buffer's mark at its position.
final MappedByteBuffer	**position**(int newPosition)	Sets this buffer's position.
	reset()	

final MappedByteBuffer		Resets this buffer's position to the previously-marked position.
final MappedByteBuffer	**rewind**()	Rewinds this buffer.
abstract MappedByteBuffer	**slice**()	Creates a new byte buffer whose content is a shared subsequence of this buffer's content.
abstract MappedByteBuffer	**slice**(int index, int length)	Creates a new byte buffer whose content is a shared subsequence of this buffer's content.

Methods declared in class ByteBuffer

alignedSlice, alignmentOffset, allocate, allocateDirect, array, arrayOffset, asCharBuffer, asDoubleBuffer, asFloatBuffer, asIntBuffer, asLongBuffer, asReadOnlyBuffer, asShortBuffer, compareTo, equals, get, get, get, get, get, get, getChar, getChar, getDouble, getDouble, getFloat, getFloat, getInt, getInt, getLong, getLong, getShort, getShort, hasArray, hashCode, isDirect, mismatch, order, order, put, put, put, put, put, put, put, put, putChar, putChar, putDouble, putDouble, putFloat, putFloat, putInt, putInt, putLong, putLong, putShort, putShort, toString, wrap, wrap

Methods declared in class Buffer

capacity, hasRemaining, isReadOnly, limit, position, remaining

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

isLoaded

```
public final boolean isLoaded()
```

Tells whether or not this buffer's content is resident in physical memory.

A return value of `true` implies that it is highly likely that all of the data in this buffer is resident in physical memory and may therefore be accessed without incurring any virtual-memory page faults or I/O operations. A return value of `false` does not necessarily imply that the buffer's content is not resident in physical memory.

The returned value is a hint, rather than a guarantee, because the underlying operating system may have paged out some of the buffer's data by the time that an invocation of this method returns.

Returns:

`true` if it is likely that this buffer's content is resident in physical memory

load

```
public final MappedByteBuffer load()
```

Loads this buffer's content into physical memory.

This method makes a best effort to ensure that, when it returns, this buffer's content is resident in physical memory. Invoking this method may cause some number of page faults and I/O operations to occur.

Returns:

This buffer

force

```
public final MappedByteBuffer force()
```

Forces any changes made to this buffer's content to be written to the storage device containing the mapped file. The region starts at index zero in this buffer and is `capacity()` bytes. An invocation of this method behaves in exactly the same way as the invocation `force(0,capacity())`.

If the file mapped into this buffer resides on a local storage device then when this method returns it is guaranteed that all changes made to the buffer since it was created, or since this method was last invoked, will have been written to that device.

If the file does not reside on a local device then no such guarantee is made.

If this buffer was not mapped in read/write mode (`FileChannel.MapMode.READ_WRITE`) then invoking this method may have no effect. In particular, the method has no effect for buffers mapped in read-only or private mapping modes. This method may or may not have an effect for implementation-specific mapping modes.

Returns:

This buffer

Throws:

`UncheckedIOException` - If an I/O error occurs writing the buffer's content to the storage device containing the mapped file

force

```
public final MappedByteBuffer force(int index,
                                    int length)
```

Forces any changes made to a region of this buffer's content to be written to the storage device containing the mapped file. The region starts at the given `index` in this buffer and is `length` bytes.

If the file mapped into this buffer resides on a local storage device then when this method returns it is guaranteed that all changes made to the selected region buffer since it was created, or since this method was last invoked, will have been written to that device. The force operation is free to write bytes that lie outside the specified region, for example to ensure that data blocks of some device-specific granularity are transferred in their entirety.

If the file does not reside on a local device then no such guarantee is made.

If this buffer was not mapped in read/write mode (`FileChannel.MapMode.READ_WRITE`) then invoking this method may have no effect. In particular, the method has no effect for buffers mapped in read-only or private mapping modes. This method may or may not have an effect for implementation-specific mapping modes.

Parameters:

`index` - The index of the first byte in the buffer region that is to be written back to storage; must be non-negative and less than `capacity()`

`length` - The length of the region in bytes; must be non-negative and no larger than `capacity() - index`

Returns:

This buffer

Throws:

`IndexOutOfBoundsException` - if the preconditions on the index and length do not hold.

`UncheckedIOException` - If an I/O error occurs writing the buffer's content to the storage device containing the mapped file

Since:

13

position

`public final MappedByteBuffer position(int newPosition)`

Sets this buffer's position. If the mark is defined and larger than the new position then it is discarded.

Overrides:

`position` in class `ByteBuffer`

Parameters:

`newPosition` - The new position value; must be non-negative and no larger than the current limit

Returns:

This buffer

Since:

9

limit

`public final MappedByteBuffer limit(int newLimit)`

Sets this buffer's limit. If the position is larger than the new limit then it is set to the new limit. If the mark is defined and larger than the new limit then it is discarded.

Overrides:

`limit` in class `ByteBuffer`

Parameters:

`newLimit` - The new limit value; must be non-negative and no larger than this buffer's capacity

Returns:

This buffer

Since:

9

mark

`public final MappedByteBuffer mark()`

Sets this buffer's mark at its position.

Overrides:

`mark` in class `ByteBuffer`

Returns:

This buffer

Since:

9

reset

`public final MappedByteBuffer reset()`

Resets this buffer's position to the previously-marked position.

Invoking this method neither changes nor discards the mark's value.

Overrides:

`reset` in class `ByteBuffer`

Returns:

This buffer

Since:

9

clear

`public final MappedByteBuffer clear()`

Clears this buffer. The position is set to zero, the limit is set to the capacity, and the mark is discarded.

Invoke this method before using a sequence of channel-read or *put* operations to fill this buffer. For example:

```
    buf.clear();     // Prepare buffer for reading
    in.read(buf);    // Read data
```

This method does not actually erase the data in the buffer, but it is named as if it did because it will most often be used in situations in which that might as well be the case.

Overrides:

`clear` in class `ByteBuffer`

Returns:

This buffer

Since:

9

flip

`public final MappedByteBuffer flip()`

Flips this buffer. The limit is set to the current position and then the position is set to zero. If the mark is defined then it is discarded.

After a sequence of channel-read or *put* operations, invoke this method to prepare for a sequence of channel-write or relative *get* operations. For example:

```
    buf.put(magic);    // Prepend header
    in.read(buf);      // Read data into rest of buffer
    buf.flip();        // Flip buffer
    out.write(buf);    // Write header + data to channel
```

This method is often used in conjunction with the `compact` method when transferring data from one place to another.

Overrides:

`flip` in class `ByteBuffer`

Returns:

This buffer

Since:

9

rewind

`public final MappedByteBuffer rewind()`

Rewinds this buffer. The position is set to zero and the mark is discarded.

Invoke this method before a sequence of channel-write or *get* operations, assuming that the limit has already been set appropriately. For example:

```
    out.write(buf);    // Write remaining data
    buf.rewind();      // Rewind buffer
    buf.get(array);    // Copy data into array
```

Overrides:

`rewind` in class `ByteBuffer`

Returns:

This buffer

Since:

9

slice

```
public abstract MappedByteBuffer slice()
```

Creates a new byte buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of bytes remaining in this buffer, its mark will be undefined, and its byte order will be `BIG_ENDIAN`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Reading bytes into physical memory by invoking `load()` on the returned buffer, or writing bytes to the storage device by invoking `force()` on the returned buffer, will only act on the sub-range of this buffer that the returned buffer represents, namely `[position(),limit())`.

Specified by:

`slice` in class `ByteBuffer`

Returns:

The new byte buffer

Since:

17

See Also:

`ByteBuffer.alignedSlice(int)`

slice

```
public abstract MappedByteBuffer slice(int index,
                                       int length)
```

Creates a new byte buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at position `index` in this buffer, and will contain `length` elements. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be `length`, its mark will be undefined, and its byte order will be `BIG_ENDIAN`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Reading bytes into physical memory by invoking `load()` on the returned buffer, or writing bytes to the storage device by invoking `force()` on the returned buffer, will only act on the sub-range of this buffer that the returned buffer represents, namely `[index,index+length)`, where `index` and `length` are assumed to satisfy the preconditions.

Specified by:

`slice` in class `ByteBuffer`

Parameters:

`index` - The position in this buffer at which the content of the new buffer will start; must be non-negative and no larger than `limit()`

`length` - The number of elements the new buffer will contain; must be non-negative and no larger than `limit() - index`

Returns:

The new buffer

Since:

17

duplicate

```
public abstract MappedByteBuffer duplicate()
```

Creates a new byte buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, and mark values will be identical to those of this buffer, and its byte order will be `BIG_ENDIAN`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

`duplicate` in class `ByteBuffer`

Returns:

The new byte buffer

Since:

17

compact

```
public abstract MappedByteBuffer compact()
```

Compacts this buffer *(optional operation)*.

The bytes between the buffer's current position and its limit, if any, are copied to the beginning of the buffer. That is, the byte at index p = `position()` is copied to index zero, the byte at index p + 1 is copied to index one, and so forth until the byte at index `limit()` - 1 is copied to index n = `limit()` - 1 - p. The buffer's position is then set to $n+1$ and its limit is set to its capacity. The mark, if defined, is discarded.

The buffer's position is set to the number of bytes copied, rather than to zero, so that an invocation of this method can be followed immediately by an invocation of another relative *put* method.

Invoke this method after writing data from a buffer in case the write was incomplete. The following loop, for example, copies bytes from one channel to another via the buffer `buf`:

```
buf.clear();          // Prepare buffer for use
while (in.read(buf) >= 0 || buf.position != 0) {
    buf.flip();
    out.write(buf);
    buf.compact();    // In case of partial write
}
```

Specified by:

`compact` in class `ByteBuffer`

Returns:

This buffer

Throws:

`ReadOnlyBufferException` - If this buffer is read-only

Since:

17

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA

94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.